



Group Assignment 1

Authentication and Verification in Distributed Zero Trust Architectures

Group B

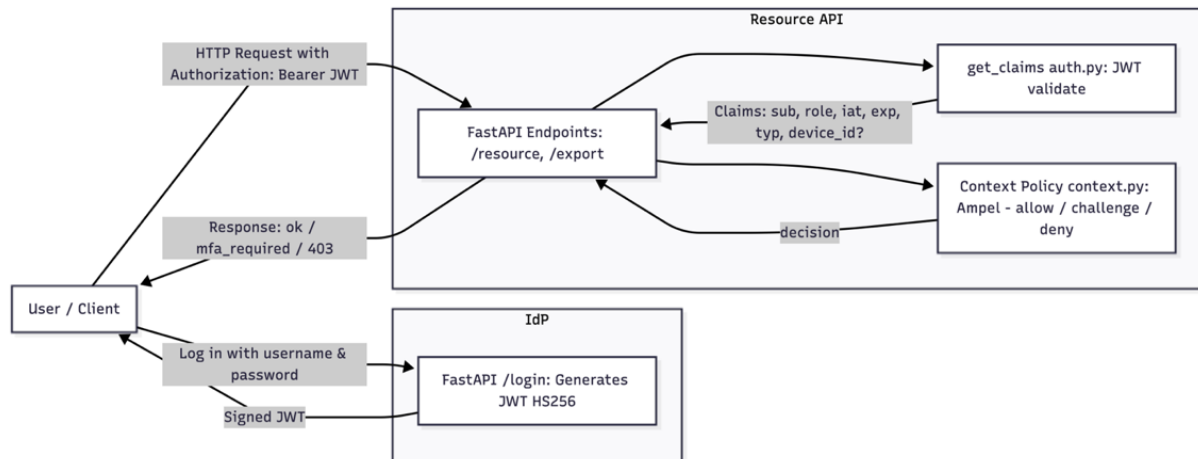
Author:	Barcikowska Anna Malgorzata, Kallioinen Jimi Eemil, Stähli Thomas, Gomez Pereanez Carlos Andres
Study Program:	SDA4 - Hands-on Architecture (AS25)
Semester:	7 th semester
Supervisor:	Sebastian Höhn
Date:	10.11.2025

Contents

1	Architecture Analysis	2
1.1	Before-Architecture IdP + resource_api	2
1.2	New-Architecture IdP + resource_api	2
1.3	Before-Architecture Local_service	3
1.4	New Architecture Local_service	3
1.5	Centralized vs. Decentralized Verification	4
2	Local Service	4
2.1	Architecture and Changes	4
2.2	Additional Verification Logic	4
3	Identity Provider (IdP)	5
4	Resource API	6
4.1	Policy Implementation and Contextual Verification Logic	6
4.2	Additional Verification Logic Explained	6
4.2.1	Tier 1: Admin-Specific Access Control	6
4.2.2	Tier 2: Global Risk Mitigation (Non-Admins)	7
4.2.3	Tier 3: Path and Context Restrictions (Specific Guardrails)	7
4.2.4	Tier 4: Default Outcome	7
5	Zero Trust principles in our extensions	8
5.1	1. Dynamic and Context-Based Access Decisions (Tenet 4 & 6)	8
5.2	2. Device Integrity and Trust Evaluation (Tenet 5)	8
5.3	Per-Session Access and Least Privilege (Tenet 3)	8
5.4	Continuous Authorization and Verification (Tenet 6)	8
5.5	Risk-Based Access Decisions (Extension of Tenet 4)	8
5.6	Foundational Security and Continuous Monitoring (Tenets 1, 2 & 7 – Not Implemented)	8
6	Sources	9

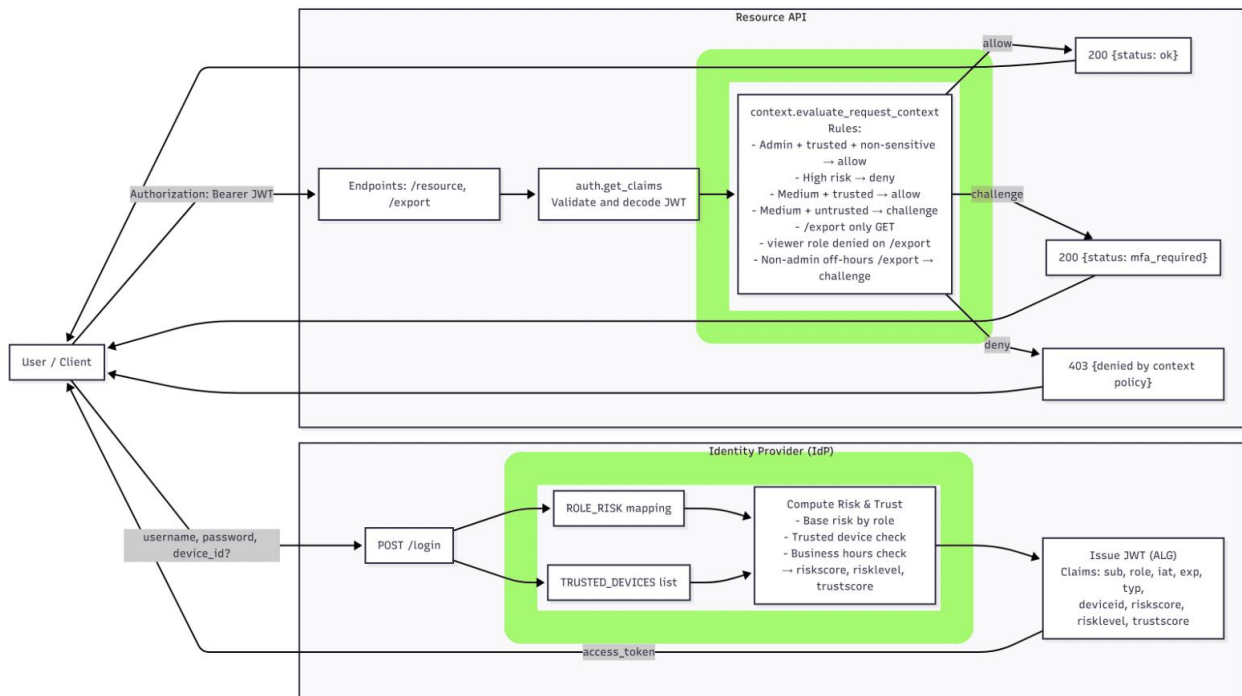
1 Architecture Analysis

1.1 Before-Architecture IdP + resource_api



In the initial architecture of the Identity Provider (IdP) and Resource API, the system followed a basic authentication and token validation flow without any contextual or risk-based checks. The IdP handles user authentication through a simple /login endpoint. When a user provided valid credentials, the IdP generated a JWT containing only basic claims — such as the username (sub), role, issue time, expiration, and token type. There was no evaluation of device, risk, or business-hour context. The token was simply signed with a shared secret and returned to the user. The Resource API then accepted this token as a Bearer JWT in each request. It validated the signature and decoded the claims, allowing access to the resource endpoints like /resource or /export. The only implemented rule was a minimal check: If a non-admin user accessed the sensitive /export endpoint -> they were challenged (simulating a potential MFA). Otherwise, access was allowed by default.

1.2 New-Architecture IdP + resource_api



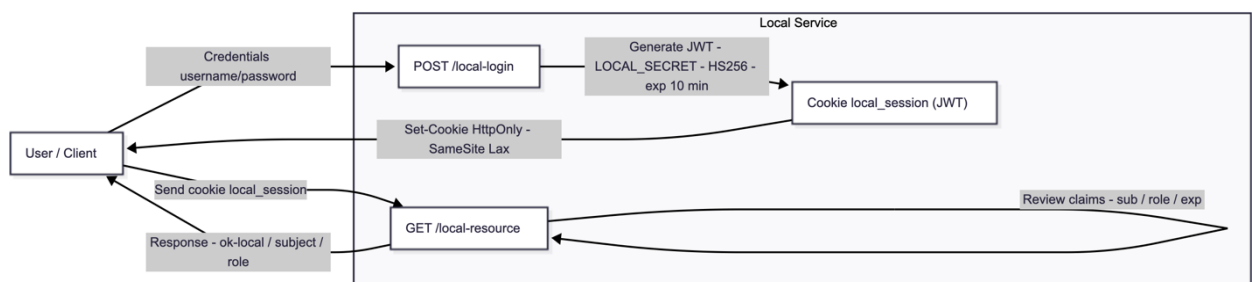
In the new architecture of the Identity Provider (IdP) and Resource API, the key enhancement is the integration of context-based policy evaluation.

When a user logs in through the IdP, their credentials and device information are processed. The IdP then calculates a risk and trust score based on several factors — including the user's role, whether the device is trusted, and if the login occurs during business hours. These results (riskscore, risklevel, trustscore) are added as claims into a JWT, which is then issued to the client.

When accessing the Resource API, the JWT is validated and its claims are evaluated by the context evaluation module. This module enforces granular access rules — for example, trusted admins accessing non-sensitive resources are allowed, while high-risk users are denied or challenged with step-up authentication (MFA). Specific rules also control access to sensitive endpoints like /export, restricting viewer roles or non-admin users during off-hours.

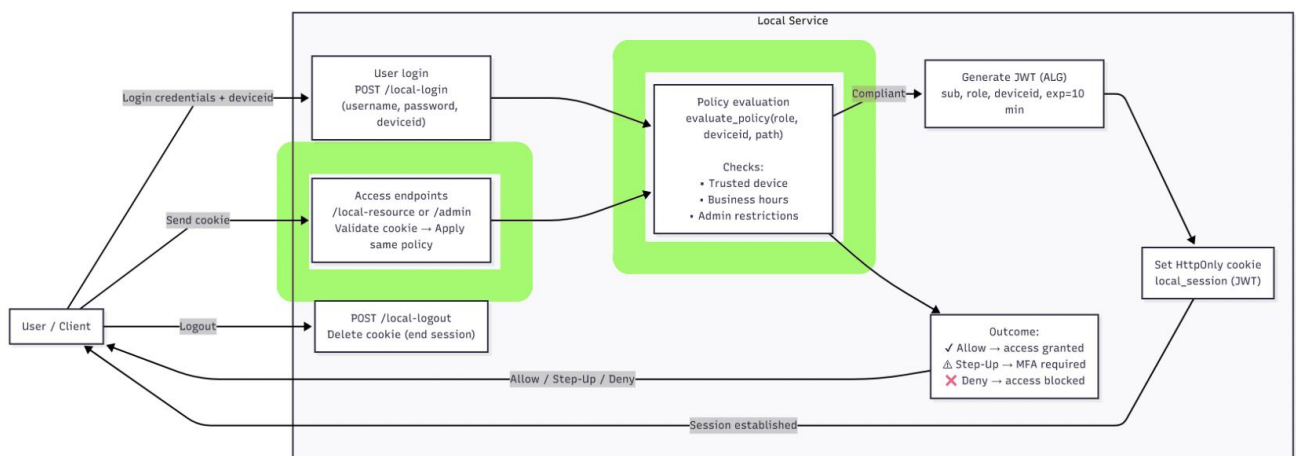
Each request is therefore dynamically assessed, ensuring that access decisions are continuously verified and aligned with Zero Trust principles.

1.3 Before-Architecture Local_service



The decentralized model works a bit differently from IdP. The Local Service performs authentication on its own without contacting the IdP. It validates the user's credentials locally and issues a short-lived JWT stored in a secure cookie. When the user accesses a protected resource, the Local Service verifies the token from the cookie and grants access if it's valid. This setup works independently and is faster but does not provide the same level of central control or contextual awareness as the IdP-based model.

1.4 New Architecture Local_service



In the new architecture, the most important addition is the policy evaluation. This component checks the trusted device, business hours, and admin restrictions for every login or access attempt. After the user logs in with valid credentials, the policy evaluation verifies these rules. If the access is allowed, it generates a JWT, encrypted using the ALG algorithm, and stores it as an HttpOnly cookie in the local session. This cookie is then used to access the local endpoints — for example, /local-resource or /admin. Before granting access, the policy evaluation runs again to ensure that the request still complies with all rules. Finally, the user receives a response that indicates the decision — either allow, step-up (meaning additional verification required), or deny.

1.5 Centralized vs. Decentralized Verification

In our implementation, we use both a centralized verification model through the Identity Provider (IdP) and a decentralized model through the Local Service.

With the centralized model, we rely on the IdP to authenticate users, evaluate context and issue signed tokens. This gives us consistent authentication logic, easier policy management and a clear point of control for risk scoring. However, if the IdP becomes unavailable, new tokens cannot be issued, and every login adds a small amount of network latency.

The decentralized model allows services to perform their own local verification without contacting the IdP. It improves availability and reduces latency, but it also means that each service has to maintain its own authentication logic, which can lead to inconsistent policies and worse visibility of user activity.

The centralized IdP approach is in our opinion better aligned with Zero Trust principles, since it provides continuous verification and risk evaluation. The decentralized model offers flexibility and independence but sacrifices central governance and consistency.

2 Local Service

2.1 Architecture and Changes

The local service issues JWT tokens as cookies during login (/local-login) and verifies them during resource access (/local-resource) at the same place locally. We added a sensitive path (/admin) to show extra verification. A Policy Decision Point was implemented with the context factors, deviceid, business hours, role, and path. In the JWT tokens we use the following claims:

- username
- role
- issued at
- expiration time
- type
- deviceid

2.2 Additional Verification Logic

During Login, a Policy Enforcement Point evaluates the provided claims based on the Policy Decision Point and decides whether the Login is allowed, denied or a step-up process is needed. The step up process could trigger a MFA for example. If the Login is allowed, the JWT token is provided.

During Resource access, the JWT token gets decoded, and a Policy Enforcement Point evaluates the claims again based on the Policy Decision Point.

The Policy Decision Point checks whether the provided deviceid is in the list of trusted devices, and if the request is during business hours. For sensitive paths like /admin, it checks additionally the role and if the deviceid is in the list of admin trusted devices.

```
# Policy Decision Point
def evaluate_policy(role: str, deviceid: str, path: str) -> str:
    now = datetime.now(TZ)

    if path == "/admin":
        if role != "admin":
            return "deny"
        if deviceid not in ADMIN_TRUSTED_DEVICES:
            return "step_up"
        if not (BUSINESS_HOURS_START <= now.hour < BUSINESS_HOURS_END):
            return "step_up"
    else:
        if deviceid not in TRUSTED_DEVICES:
            return "deny"
        if not (BUSINESS_HOURS_START <= now.hour < BUSINESS_HOURS_END):
            return "step_up"

    return "allow"
```

3 Identity Provider (IdP)

1. Architecture Overview

The IdP acts as a **central trust authority** and exposes a single /login endpoint. When a user logs in, the IdP authenticates the username and password, then generates a JWT signed with a shared secret.

The token contains standard claims (subject, issued time, expiry) and several new claims added during this implementation:

- role - user privilege level (analyst, contractor, and newly added admin)
- deviceid - identifier of the authenticating device
- riskscore - risk value from 1-10
- risklevel - risk label ("low", "medium", "high")
- trustscore - inverse trust metric from 0-100

These claims let other services decide whether to allow access based on the current context such as user role, device and time instead of relying only on who the user is.

2. Contextual Verification Logic (Zero Trust)

The IdP combines identity, device, and time to calculate a risk score dynamically for each login attempt.

We also implemented the optional trust-scoring enhancement, where the IdP calculates a dynamic score based on identity (role), device trust and context (time), returning both a numeric riskscore and a trustscore.

Role-based risk	Device-based check	Time-based check
Admin (8), Contractor (5), Analyst (2)	+2 risk if the device is not trusted	+2 risk for logins outside of business hours - 07:00-19:00
<pre>ROLE_RISK = { "analyst": 2, "contractor": 5, "admin": 8, }</pre>	<pre>#trusted device check if not trusted_device: risk += 2</pre>	<pre>hour = now.hour if hour < 7 or hour > 19: risk += 2</pre>

Risk is capped at 10 and later converted into trust. This provides an adaptive "trust score" that changes depending on user and context.

For example:

- **Admin** logging in during business hours → risk 10, trust 20 (high risk)

<pre>http://localhost:8001/docs curl -X 'POST' \ 'http://localhost:8001/login' \ -H 'accept: application/json' \ -H 'Content-Type: application/json' \ -d '{ "username": "admin", "password": "admin", "device_id": "admin-laptop" }'</pre>	<pre>https://www.jwt.io/ { "sub": "admin", "role": "admin", "iat": 1761474521, "exp": 1761476321, "typ": "access", "deviceid": "admin-laptop", "riskscore": 8, "risklevel": "high", "trustscore": 20 }</pre>
--	---

- **Contractor** during business hours → risk 5, trust 50 (medium risk)

```

http://localhost:8001/docs
curl -X 'POST' \
  'http://localhost:8001/login' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "username": "contractor",
    "password": "contractor",
    "device_id": "contractor-laptop"
  }'

https://www.jwt.io/
{
  "sub": "contractor",
  "role": "contractor",
  "iat": 1761474556,
  "exp": 1761476356,
  "typ": "access",
  "deviceid": "contractor-laptop",
  "riskscore": 5,
  "risklevel": "medium",
  "trustscore": 50
}

```

Each token that the IdP issues now includes a calculated trust score based on the user, device, and context. This means the system no longer just checks if a login is correct, it also measures how trustworthy that login is. The IdP has moved from simple authentication to context-aware verification, where every login is evaluated continuously instead of being trusted automatically.

4 Resource API

4.1 Policy Implementation and Contextual Verification Logic

The Resource API acts as the Policy Enforcement Point (PEP), protecting the application's resources (like the sensitive /export endpoint). The core of this service is the Contextual Access Policy (context.py), which serves as the Policy Decision Point (PDP). This policy consumes the rich claims issued by the Identity Provider (IdP) and applies a series of logical rules to determine the final access decision: allow, challenge or deny.

Policy Architecture and Execution Flow

The policy is structured in sequential tiers, and execution stops immediately upon the first rule that returns a decision.

- **Tier 1 (Rules 1a & 1b):** Admin-Specific Access Control
- **Tier 2 (Rules 2a - 2c):** Global Risk Mitigation (Non-Admins)
- **Tier 3 (Rules 3a - 3c):** Path and Context Restrictions
- **Tier 4 (Default):** Low-Risk Default Allow

4.2 Additional Verification Logic Explained

The policy's primary goal is to shift security from a simple binary check ("Is the password correct?") to a continuous adaptive assessment ("Is this a safe context?").

4.2.1 Tier 1: Admin-Specific Access Control

This tier handles high-privilege users, immediately resolving their access based on path and device trust, bypassing general risk scores.

1. **Sensitive Path & Trusted Device:** The request is set to **CHALLENGE**. This enforces a step-up MFA for high-stakes actions, even on trusted machines.
2. **Sensitive Path & Untrusted Device:** The request is set to **DENY**. Access to sensitive resources from an unverified source is considered an unacceptable risk.
3. **Non-Sensitive Path & Trusted Device:** The request is **ALLOWED**. This prioritizes operational efficiency for standard admin tasks in a verified context.
4. **Non-Sensitive Path & Untrusted Device:** The request is set to **CHALLENGE**. This requires verification when context (device trust) is missing.

4.2.2 Tier 2: Global Risk Mitigation (Non-Admins)

This tier governs all users not covered by the admin rules, primarily filtering by the `risk_level`.

1. **Rule 2a:** Universal Deny (HIGH Risk): Any request with a "HIGH" risk level is immediately DENIED.
2. **Rules 2b & 2c:** Handling MEDIUM Risk:
 - **Exemption (Rule 2b):** If the risk is "MEDIUM", the device is trusted, and the path is non-sensitive, access is ALLOWED.
 - **Default Challenge (Rule 2c):** All other "MEDIUM" risk requests (untrusted device, sensitive path, or both) result in a CHALLENGE (MFA).

4.2.3 Tier 3: Path and Context Restrictions (Specific Guardrails)

These rules apply specific security measures based on the resource or context, before the final default allow.

1. **Rule 3a:** HTTP Method Restriction: If the path is sensitive and the HTTP method is not "GET" (e.g., POST, DELETE), the request is DENIED to protect data integrity.
2. **Rule 3b:** Least Privilege Denial for /export: If the path is /export and the role is "viewer", the request is DENIED, enforcing the Principle of Least Privilege (PoLP).
3. **Rule 3c:** Time-Based Challenge (Non-Admins): If the path is sensitive, the user is not an admin, and the request is outside BUSINESS_HOURS, a CHALLENGE is issued.

4.2.4 Tier 4: Default Outcome

If a non-admin request passes all preceding rules (meaning the risk is implicitly **LOW**), the system falls through to the final line and **ALLOWS** access.

5 Zero Trust principles in our extensions

In our extensions of the two authentication types – the IdP with Resource API and the Local Service – we applied several Zero Trust principles to make resource access more secure and context aware. The goal was that no access is automatically trusted, but that every request is continuously evaluated based on its context.

5.1 1. Dynamic and Context-Based Access Decisions (Tenet 4 & 6)

A central principle implemented in both systems is **context-based access control**. In both setups, each request is checked based on the user's role, the device used, and the time of access. Depending on these factors, access is either allowed, denied, or requires an additional verification step. In the IdP + Resource API, this is achieved through a calculated risk score included in the token, while in the Local Service the function *evaluate_policy* directly evaluates role, device, and time for each request.

5.2 2. Device Integrity and Trust Evaluation (Tenet 5)

The device verification principle was applied in both solutions. In the IdP + Resource API, the provided *device_id* is checked to determine whether the device is trusted, and unrecognized devices increase the user's risk score. In the Local Service, we introduced an allowlist of approved devices (for example, *lab-1* or *office-pc*, and a separate list for admin devices. This ensures that only known and approved devices can access resources and that administrative actions require additional trust.

5.3 Per-Session Access and Least Privilege (Tenet 3)

The principle of limited session lifetime was also implemented. In the IdP + Resource API, tokens are valid for 30 minutes, while in the Local Service they expire after only 10 minutes. This reduces the risk of stolen or inactive sessions and ensures that users must reauthenticate regularly. Each session grants access only to specific endpoints; for example, in the Local Service, accessing */admin* requires both the admin role and an approved admin device.

5.4 Continuous Authorization and Verification (Tenet 6)

Both systems continuously verify authorization rather than relying on a single login event. In the Resource API, every request triggers *evaluate_request_context*, while in the Local Service, *evaluate_policy* is used to recheck each access attempt. This ensures that trust is never permanent and that every action is reassessed according to current conditions.

5.5 Risk-Based Access Decisions (Extension of Tenet 4)

Only in the IdP + Resource API did we implement a risk-based decision model. The IdP calculates a risk score and level using multiple factors such as user role, trusted device status, and time of day. The Resource API then uses this score to decide whether to allow, deny, or challenge the request (for example, MFA required). This adaptive mechanism reflects a key Zero Trust concept—access decisions based on continuous risk assessment.

5.6 Foundational Security and Continuous Monitoring (Tenets 1, 2 & 7 – Not Implemented)

The principles of treating all systems as protected resources, securing all communication, and continuously monitoring activity were not implemented in our project. Our focus was on authentication and authorization rather than full resource classification, encrypted communication, or telemetry collection. It would have gone beyond the scope The FastAPI services communicated locally over HTTP without TLS, and no continuous monitoring of user or device behavior was performed. In a real Zero Trust environment, these aspects would be essential to ensure that all data sources and communications are secured end-to-end and that system activity is constantly analyzed to detect threats and adapt policies dynamically.

6 Sources

- Rose, S., Borchert, O., Mitchell, S., & Connelly, S. (2020, August). Zero Trust Architecture (NIST Special Publication 800-207). U.S. Department of Commerce, National Institute of Standards and Technology.
<https://doi.org/10.6028/NIST.SP.800-207>